

SQL Queries

Databases - Computer Science

1. Logical evaluation order

A SELECT is written SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, but evaluated FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. This explains why a column alias defined in SELECT cannot be used in WHERE, yet can be used in ORDER BY.

2. Filtering and grouping

WHERE filters individual rows before grouping; HAVING filters groups after aggregation. Aggregates - COUNT, SUM, AVG, MIN, MAX - ignore NULL, with the exception of COUNT(*), which counts rows. Any non-aggregated column in the SELECT list must appear in GROUP BY.

3. Joins in SQL

INNER JOIN keeps only matching pairs. LEFT and RIGHT OUTER JOIN keep unmatched rows from one side, filling the other with NULL. FULL OUTER keeps both. A missing join predicate silently produces a Cartesian product, which is the usual cause of a query that returns far more rows than expected.

4. NULL and three-valued logic

Comparison with NULL yields UNKNOWN, not true or false, so WHERE $x = \text{NULL}$ never matches - IS NULL is required. UNKNOWN in a WHERE clause behaves as false, which is why NOT IN against a subquery containing a NULL returns no rows at all.